



UNIVERSITÉ BOURGOGNE EUROPE

DÉPARTEMENT INFORMATIQUE, ÉLECTRONIQUE, MÉCANIQUE (IEM)

Travaux Pratiques SR2

Systemes et Réseaux 2

Mise en place d'un réseau d'entreprise et infrastructure de services

Groupe 2

Ryan LAWSON

Jessy NAHIMANA

Youcef M'SILI

Numéro du serveur : 6

Adresse IEM : **172.31.20.120**

Février 2026

Table des matières

1	Introduction	4
1.1	Matériel et environnement	4
1.2	Consignes de sécurité	4
2	Adressage et routage (TP1)	4
2.1	Installation de Debian 13 Trixie	4
2.1.1	Connexion à l'iLO	4
2.1.2	Boot réseau PXE	4
2.1.3	Schéma de partitionnement	4
2.2	Plan d'adressage collaboratif	5
2.2.1	Collaboration entre groupes	5
2.2.2	Réseau IEM	5
2.2.3	Réseau d'interconnexion	6
2.2.4	Réseau privé de l'agence	6
2.3	Configuration des interfaces réseau	6
2.3.1	Fichier /etc/network/interfaces	6
2.3.2	Vérification avec ip a	7
2.4	Implantation et tests de connectivité	7
2.4.1	Test du réseau d'interconnexion	7
2.4.2	Test du réseau privé	8
2.5	Activation du routage et routes statiques	9
2.5.1	Activation du forwarding IP	9
2.5.2	Routes statiques vers les autres groupes	9
2.5.3	Validation du routage	9
2.6	Règle NAT pour l'accès Internet (Question 8)	9
2.7	Serveur DHCP (Question 9)	10
2.7.1	Installation et interface d'écoute	10
2.7.2	Configuration du sous-réseau et réservation MAC	10
2.7.3	Logs spécifiques	10
2.8	Sauvegarde automatique avec rsync (Question 10)	11
2.8.1	Génération des clés SSH	11
2.8.2	Mise en place de la tâche cron	11
2.9	Conclusion du TP1	11
3	Installation d'un serveur DNS (TP2)	12
3.1	Objectifs du TP2	12
3.2	Interrogation DNS – Phase d'apprentissage	12
3.2.1	Installation des outils	12
3.2.2	Résolution de www.u-bourgogne.fr	12
3.2.3	Interprétation des résultats	13
3.2.4	Changement de serveur DNS avec dig	13
3.2.5	Recherche des enregistrements NS (Name Servers)	14
3.2.6	Parcours de la résolution avec +trace	14
3.2.7	Fichiers systèmes liés au DNS	14
3.3	Installation et configuration de BIND9	14
3.3.1	Installation du paquet	14
3.3.2	Arborescence des fichiers de configuration	15

3.4	Création de la zone directe (nom → IP)	15
3.4.1	Fichier db.agence.sourire	15
3.5	Création de la zone inverse (IP → nom)	16
3.5.1	Fichier db.10.2.6	16
3.6	Déclaration des zones dans named.conf.local	17
3.7	Configuration des forwarders et options globales	17
3.7.1	Fichier named.conf.options	17
3.8	Vérification de la configuration	18
3.9	Démarrage et activation du service	18
3.10	Tests de résolution depuis le serveur	19
3.10.1	Configuration du résolveur local	19
3.10.2	Résolution directe	19
3.10.3	Résolution inverse	19
3.10.4	Tests des forwarders	19
3.11	Intégration avec le serveur DHCP	19
3.12	Résolution inter-groupes	20
3.12.1	Ajout d'une zone forward vers l'agence oseille	20
3.12.2	Rechargement de BIND9 et test	21
3.13	Conclusion du TP2	21
4	Transition vers le TP3	21
5	Installation et configuration d'applications critiques (TP3)	21
5.1	Serveur web Apache	22
5.1.1	Installation et validation de base	22
5.1.2	Modification du DNS pour les nouveaux noms	22
5.1.3	Création du site pour web1 (alias /web1)	23
5.1.4	Création du site pour web2 (VirtualHost)	23
5.1.5	Publication de pages personnelles (public_html) pour web3	24
5.1.6	Installation de PHP	24
5.2	Systèmes de gestion de bases de données	24
5.2.1	Création de la partition /bd	24
5.2.2	Installation et configuration de MariaDB	25
5.2.3	Installation et configuration de PostgreSQL	25
5.3	PDO – Affichage des tables depuis PHP	26
5.4	Sauvegarde des bases de données	27
5.5	Sécurisation du serveur (iptables)	27
5.5.1	Matrice de filtrage	27
5.5.2	Script de filtrage	28
5.5.3	Script “allow all” pour le débogage	29
5.5.4	Automatisation au démarrage	29
5.6	Droits sudo pour web1	29
5.7	Conclusion du TP3	30
6	Conclusion générale des TP1 à TP3	30

7	Authentification et partage de ressources (TP4)	30
7.1	Définition du DIT (Directory Information Tree)	30
7.2	Installation et configuration d'OpenLDAP	31
7.2.1	Installation des paquets	31
7.2.2	Configuration initiale	31
7.2.3	Démarrage et test	31
7.3	Ajout du schéma Samba	32
7.4	Peuplement de l'annuaire avec smbldap-tools	32
7.4.1	Installation des outils	32
7.4.2	Configuration de smbldap-tools	32
7.4.3	Populate – création des OU et des entrées de base	32
7.5	Création manuelle des utilisateurs (contournement)	33
7.5.1	Hash des mots de passe	33
7.5.2	Fichier LDIF pour user1	33
7.5.3	Ajout dans LDAP	33
7.6	Configuration de Samba pour LDAP	34
7.7	Authentification système via LDAP (libnss-ldapd)	34
7.7.1	Installation	34
7.7.2	Vérification des fichiers de configuration	34
7.8	Création des partages Samba	35
7.8.1	Répertoires physiques	35
7.8.2	Définition des partages dans smb.conf	35
7.8.3	Redémarrage de Samba	35
7.9	Validation des accès	35
7.9.1	Installation de smbclient	35
7.9.2	Définition des mots de passe Samba	35
7.9.3	Test du partage commun	36
7.9.4	Test du partage privé	36
7.10	Sécurisation – Adaptation d'iptables	36
7.11	Sauvegarde de l'annuaire	37
7.12	Conclusion du TP4	37
8	Conclusion générale	37

1 Introduction

L'objectif de cette série de travaux pratiques est la mise en place d'un réseau d'entreprise complet ainsi qu'une infrastructure de services. Chaque groupe, considéré comme une agence, doit interconnecter son réseau avec ceux des autres groupes via un réseau dédié. Ce rapport détaille l'ensemble des étapes réalisées, des choix techniques effectués et des validations expérimentales menées.

1.1 Matériel et environnement

- **Serveur** : HPE ProLiant (routeur de l'agence)
- **Client** : Portable Dell pour les tests
- **OS serveur** : Debian 13 Trixie (sans interface graphique)
- **Accès distant** : iLO (<http://serv-06.iem>) et SSH

1.2 Consignes de sécurité

Conformément aux consignes, nous avons :

- Changé le mot de passe du compte `etudiant` sur l'iLO
- Défini un mot de passe non trivial pour le compte `root`
- Désactivé les accès superflus

2 Adressage et routage (TP1)

2.1 Installation de Debian 13 Trixie

2.1.1 Connexion à l'iLO

Le serveur HPE ProLiant est administré à distance via son interface iLO (Integrated Lights-Out). L'adresse d'accès est [http://serv-\[numéro-serveur\].iem](http://serv-[numéro-serveur].iem). Pour notre groupe, le serveur numéro 6 est utilisé : <http://serv-06.iem>.

Les identifiants par défaut sont :

- **Login** : `etudiant`
- **Mot de passe** : `etudiant`

Conformément aux consignes de sécurité (Encadré 1, page 2), nous avons immédiatement changé le mot de passe du compte `etudiant` sur l'iLO, ainsi que le mot de passe `root` après installation.

2.1.2 Boot réseau PXE

Depuis la console iLO, nous avons redémarré le serveur et sélectionné l'option *Network Boot (PXE)*. Le serveur a chargé l'installateur Debian 13 Trixie depuis l'infrastructure IEM.

2.1.3 Schéma de partitionnement

Un serveur doit séparer les données système des données applicatives et des logs afin de faciliter la maintenance et les sauvegardes. Nous disposons d'un disque d'environ 450 Gio. Le partitionnement retenu est le suivant :

Point de montage	Taille	Rôle
/	20 Gio	Système de base et applications
/boot/efi	1 Gio	Démarrage UEFI
/home	10 Gio	Comptes utilisateurs locaux (optionnel)
/var	30 Gio	Logs, spools, bases de données temporaires
/bd	Reste (env. 349 Gio)	Données des SGBD (MariaDB, PostgreSQL)
swap	4 Gio	Mémoire virtuelle

TABLE 1 – Partitionnement du disque

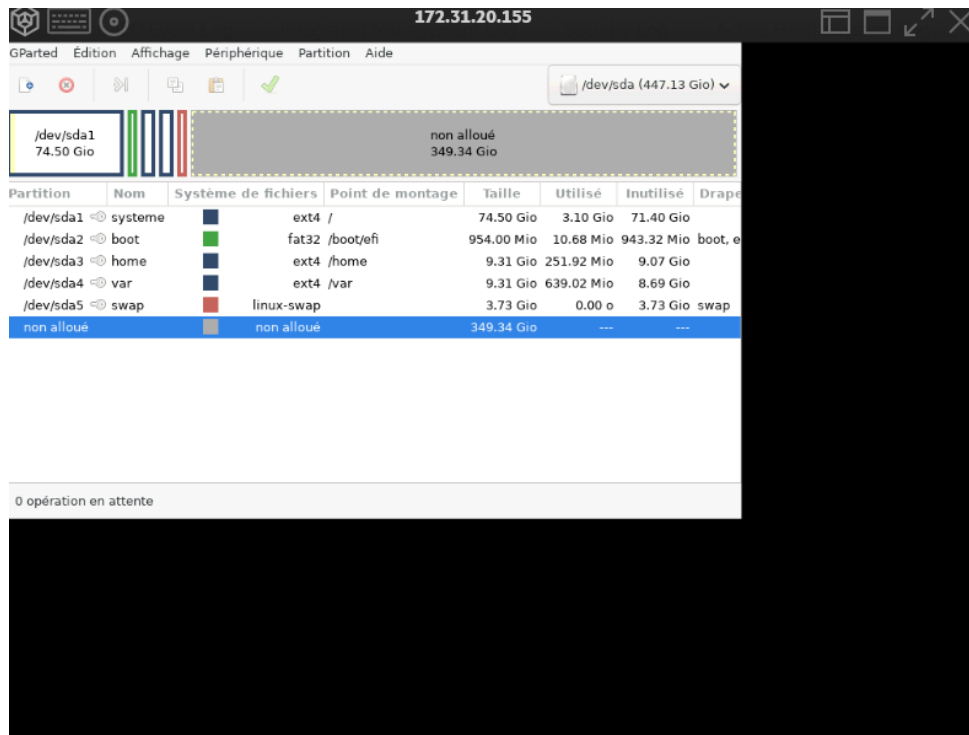


FIGURE 1 – Fenêtre GParted après création des partitions

Ce choix garantit que les logs (`/var`) ne peuvent pas saturer la racine, et que les bases de données (`/bd`) sont sur une zone dédiée.

2.2 Plan d’adressage collaboratif

2.2.1 Collaboration entre groupes

Conformément à l’encadré 2 (page 4), chaque groupe dispose de plages d’adresses réservées. Notre groupe porte le numéro **2** et notre serveur le numéro **6**.

2.2.2 Réseau IEM

Le serveur reçoit une adresse fixe dans la plage `172.31.20.12X`. Nous avons choisi : `172.31.20.120/24` ($X = 2$). La passerelle par défaut est `172.31.20.1` et le DNS `172.31.21.35`.

2.2.3 Réseau d'interconnexion

Chaque groupe dispose d'une plage de 5 adresses dans 192.168.0.0/24. Pour le groupe 2, la plage est 192.168.0.6 à 192.168.0.10. Nous avons retenu l'adresse 192.168.0.6/29 (masque /29 correspondant à 8 adresses, dont 6 utilisables).

2.2.4 Réseau privé de l'agence

Le sujet impose une adresse de la forme `x.[n° du groupe].[n° du serveur].y` avec un masque 255.255.255.0. Pour la classe A, nous choisissons la valeur conventionnelle 10. Ainsi :

- Réseau : 10.2.6.0/24
- Routeur : 10.2.6.1
- Client : 10.2.6.2

Réseau	Interface	Adresse / Masque
IEM	ens1f0	172.31.20.120/24
Interconnexion	ens1f1	192.168.0.6/29
Privé	ens1f3	10.2.6.1/24

TABLE 2 – Plan d'adressage final du groupe 2

2.3 Configuration des interfaces réseau

2.3.1 Fichier `/etc/network/interfaces`

Nous avons configuré chaque interface de façon permanente dans ce fichier.

Listing 1 – /etc/network/interfaces – Configuration statique

```
1 # Boucle locale
2 auto lo
3 iface lo inet loopback
4
5 # Interface IEM (Administration et acc s Internet)
6 auto ens1f0
7 allow-hotplug ens1f0
8 iface ens1f0 inet static
9     address 172.31.20.120
10    netmask 255.255.255.0
11    gateway 172.31.20.1
12    dns-nameservers 172.31.21.35
13
14 # Interface Interconnexion (liaison entre groupes)
15 auto ens1f1
16 allow-hotplug ens1f1
17 iface ens1f1 inet static
18     address 192.168.0.6
19     netmask 255.255.255.248    # /29
20
21 # Interface Réseau Privé (clients de l'agence)
22 auto ens1f3
23 allow-hotplug ens1f3
24 iface ens1f3 inet static
25     address 10.2.6.1
26     netmask 255.255.255.0
```

2.3.2 Vérification avec ip a

Après redémarrage du service réseau, nous obtenons la sortie suivante :

```
1 $ ip a
2 2: ens1f0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 ...
3     inet 172.31.20.120/24 brd 172.31.20.255 scope global ens1f0
4 3: ens1f1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 ...
5     inet 192.168.0.6/29 scope global ens1f1
6 4: ens1f3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 ...
7     inet 10.2.6.1/24 scope global ens1f3
```

2.4 Implantation et tests de connectivité

2.4.1 Test du réseau d'interconnexion

Nous avons utilisé le serveur de référence dont l'adresse interconnexion est 192.168.0.21.


```

root@Sourire:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens1f0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 98:f2:b3:30:72:1c brd ff:ff:ff:ff:ff:ff
    altname enp17s0f0
    altname enx98f2b330721c
    inet 172.31.20.120/24 brd 172.31.20.255 scope global ens1f0
        valid_lft forever preferred_lft forever
    inet6 fe80::9af2:b3ff:fe30:721c/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
3: eno1np0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 5c:ed:8c:ee:bd:3c brd ff:ff:ff:ff:ff:ff
    altname enp100s0f0np0
    altname enx5ced8ceebd3c
4: ens1f1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 98:f2:b3:30:72:1d brd ff:ff:ff:ff:ff:ff
    altname enp17s0f1
    altname enx98f2b330721d
    inet 192.168.0.6/29 brd 192.168.0.255 scope global ens1f1
        valid_lft forever preferred_lft forever
    inet6 fe80::9af2:b3ff:fe30:721d/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
5: ens1f2: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 98:f2:b3:30:72:1e brd ff:ff:ff:ff:ff:ff
    altname enp17s0f2
    altname enx98f2b330721e
6: eno2np1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 5c:ed:8c:ee:bd:3d brd ff:ff:ff:ff:ff:ff
    altname enp100s0f1np1
    altname enx5ced8ceebd3d
7: ens1f3: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN group default qlen 1000
    link/ether 98:f2:b3:30:72:1f brd ff:ff:ff:ff:ff:ff
    altname enp17s0f3
    altname enx98f2b330721f
    inet 10.2.6.1/24 brd 10.2.6.255 scope global ens1f3
        valid_lft forever preferred_lft forever

```

FIGURE 2 – Résultat de la commande ip a avec les trois interfaces UP

```

1 $ ping -c 4 192.168.0.21
2 PING 192.168.0.21 (192.168.0.21) 56(84) bytes of data.
3 64 bytes from 192.168.0.21: icmp_seq=1 ttl=64 time=0.8 ms
4 64 bytes from 192.168.0.21: icmp_seq=2 ttl=64 time=0.7 ms
5 64 bytes from 192.168.0.21: icmp_seq=3 ttl=64 time=0.7 ms
6 64 bytes from 192.168.0.21: icmp_seq=4 ttl=64 time=0.8 ms
7
8 --- 192.168.0.21 ping statistics ---
9 4 packets transmitted, 4 received, 0% packet loss, time 3048ms

```

2.4.2 Test du réseau privé

Le client portable Dell a été configuré avec l'adresse 10.2.6.2/24, la passerelle 10.2.6.1 et un DNS temporaire (172.31.21.35).

Depuis le serveur, nous envoyons un ping vers le client :

```

1 $ ping -c 4 10.2.6.2
2 PING 10.2.6.2 (10.2.6.2) 56(84) bytes of data.
3 64 bytes from 10.2.6.2: icmp_seq=1 ttl=64 time=1.2 ms
4 64 bytes from 10.2.6.2: icmp_seq=2 ttl=64 time=0.9 ms
5 64 bytes from 10.2.6.2: icmp_seq=3 ttl=64 time=1.1 ms
6 64 bytes from 10.2.6.2: icmp_seq=4 ttl=64 time=1.0 ms
7
8 --- 10.2.6.2 ping statistics ---
9 4 packets transmitted, 4 received, 0% packet loss, time 3004ms

```

FIGURE 3 – Ping réussi entre le serveur (10.2.6.1) et le client (10.2.6.2)

2.5 Activation du routage et routes statiques

2.5.1 Activation du forwarding IP

Le noyau Linux ne route pas les paquets par défaut. Il faut activer explicitement le paramètre `net.ipv4.ip_forward`.

```

1 $ sudo sysctl -w net.ipv4.ip_forward=1
2 $ echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf

```

2.5.2 Routes statiques vers les autres groupes

Pour que notre routeur sache joindre les réseaux privés des groupes 1, 3 et 4, nous ajoutons dans `/etc/network/interfaces` (section `ens1f1`) :

```

1 up ip route add 10.1.0.0/16 via 192.168.0.1 || true # Groupe 1
2 up ip route add 10.3.0.0/16 via 192.168.0.11 || true # Groupe 3
3 up ip route add 10.4.0.0/16 via 192.168.0.16 || true # Groupe 4

```

Le `|| true` évite que l'interface ne monte pas si la route existe déjà.

2.5.3 Validation du routage

Depuis notre client privé (10.2.6.2), nous atteignons maintenant le client du groupe 1 :

```

1 $ ping 10.1.6.2
2 PING 10.1.6.2 (10.1.6.2) 56(84) bytes of data.
3 64 bytes from 10.1.6.2: icmp_seq=1 ttl=62 time=2.3 ms

```

2.6 Règle NAT pour l'accès Internet (Question 8)

Une seule règle iptables est nécessaire pour masquer tout le trafic du réseau privé derrière l'adresse IP de l'interface IEM.

```

1 $ sudo iptables -t nat -A POSTROUTING -s 10.2.6.0/24 -o ens1f0 -j MASQUERADE

```

Pour rendre cette règle persistante, nous l'avons ajoutée dans `/etc/network/interfaces` (section `ens1f0`) :

```
1 post-up iptables -t nat -A POSTROUTING -s 10.2.6.0/24 -o ens1f0 -j
  MASQUERADE
```

```
root@Sourire:~# iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source    destination
Chain POSTROUTING (policy ACCEPT 715 packets, 109K bytes)
 pkts bytes target    prot opt in     out     source    destination
  0      0 MASQUERADE all  --  *      ens1f0  10.2.6.0/24  0.0.0.0/0
```

FIGURE 4 – Vérification de la règle NAT dans iptables

2.7 Serveur DHCP (Question 9)

2.7.1 Installation et interface d'écoute

Nous avons installé `isc-dhcp-server` et configuré l'interface d'écoute dans `/etc/default/isc-dhcp-server`

```
1 $ apt install isc-dhcp-server -y
2 $ echo 'INTERFACESv4="ens1f3"' > /etc/default/isc-dhcp-server
```

2.7.2 Configuration du sous-réseau et réservation MAC

Le fichier `/etc/dhcp/dhcpd.conf` contient la définition du réseau privé et une réservation pour le client du groupe.

```
1 subnet 10.2.6.0 netmask 255.255.255.0 {
2     range 10.2.6.100 10.2.6.200;
3     option routers 10.2.6.1;
4     option domain-name-servers 172.31.21.35;
5     option broadcast-address 10.2.6.255;
6     default-lease-time 600;
7     max-lease-time 7200;
8 }
9
10 host client-fixe {
11     hardware ethernet 4c:d7:17:16:0b:6e;    # Adresse MAC du
12     portable Dell
13     fixed-address 10.2.6.2;
14 }
```

2.7.3 Logs spécifiques

Nous avons redirigé les logs DHCP vers un fichier dédié `/var/log/dhcpd.log` en modifiant la configuration de `rsyslog` :

```

1 $ echo "local7.* /var/log/dhcpd.log" >> /etc/rsyslog.d/50-default.
   conf
2 $ systemctl restart rsyslog

```

```

root@Sourire:~# cat /var/lib/dhcp/dhcpd.leases
# The format of this file is documented in the dhcpd.leases(5) manual page.
# This lease file was written by isc-dhcp-4.4.3-P1

# authoring-byte-order entry is generated, DO NOT DELETE
authoring-byte-order little-endian;

lease 10.2.6.10 {
    starts 2 2026/04/14 14:04:29;
    ends 2 2026/04/14 14:14:29;
    tstp 2 2026/04/14 14:14:29;
    cltt 2 2026/04/14 14:04:29;
    binding state free;
    hardware ethernet 4c:d7:17:16:0b:6e;
    uid "\001L\327\027\026\013n";
}
server-duid "\000\001\000\0011|\016\375\230\362\2630r\037";

```

FIGURE 5 – Contenu du fichier `/var/lib/dhcp/dhcpd.leases` après attribution d'une IP au client

2.8 Sauvegarde automatique avec rsync (Question 10)

2.8.1 Génération des clés SSH

Pour automatiser la sauvegarde vers le serveur `aragon.iem` sans saisie de mot de passe, nous avons généré une paire de clés et copié la clé publique.

```

1 $ ssh-keygen -t rsa -b 4096 -N "" -f ~/.ssh/id_rsa
2 $ ssh-copy-id etudiant@aragon.iem

```

2.8.2 Mise en place de la tâche cron

La sauvegarde du dossier `/etc` est programmée chaque jour à 15h.

```

1 $ crontab -e
2 0 15 * * * rsync -avz -e ssh --delete /etc/ etudiant@aragon.iem:~/
   sauvegarde_$(hostname)/

```

2.9 Conclusion du TP1

Toutes les fonctionnalités demandées pour le premier TP ont été réalisées :

- Installation et partitionnement de Debian 13.
- Plan d'adressage collaboratif (IEM, interconnexion, privé).
- Configuration statique des trois interfaces réseau.
- Activation du routage IP et des routes statiques vers les autres groupes.
- Mise en place du NAT pour l'accès Internet.
- Serveur DHCP avec réservation d'adresse par MAC.
- Sauvegarde automatique quotidienne de `/etc` via rsync et cron.

Cette infrastructure de base est maintenant prête à accueillir les services DNS, web et d'annuaire.

```

root@Sourire:~# crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
@reboot /root/filtrage.sh
0 15 * * * rsync -avz /etc jn799447@aragon.iem:~/backup/

```

FIGURE 6 – Visualisation de la ligne cron ajoutée

3 Installation d'un serveur DNS (TP2)

3.1 Objectifs du TP2

Le second travail pratique vise à installer et configurer un serveur DNS faisant autorité pour notre domaine d'agence. Il se décompose en deux phases :

1. Interrogation de serveurs DNS existants avec les commandes `host`, `dig` et `nslookup` pour comprendre le fonctionnement du protocole.
2. Installation et configuration de BIND9 (Berkeley Internet Name Domain) pour gérer notre propre zone DNS.

Ce service est indispensable pour permettre la résolution de noms interne (ex : `serveur6.agence.sourire`) et externe (via des forwarders).

3.2 Interrogation DNS – Phase d'apprentissage

3.2.1 Installation des outils

Avant toute manipulation, nous avons installé le paquet `dnsutils` qui contient les commandes `dig`, `host` et `nslookup`.

```

1 $ apt update
2 $ apt install dnsutils -y

```

3.2.2 Résolution de `www.u-bourgogne.fr`

Nous avons utilisé les trois commandes sur le nom `www.u-bourgogne.fr` et comparé les résultats.

```

1 $ host www.u-bourgogne.fr
2 www.u-bourgogne.fr is an alias for web.u-bourgogne.fr.
3 web.u-bourgogne.fr has address 193.50.48.24
4
5 $ dig www.u-bourgogne.fr +short
6 193.50.48.24
7
8 $ nslookup www.u-bourgogne.fr
9 Server:          172.31.21.35
10 Address:         172.31.21.35#53
11
12 Non-authoritative answer:
13 www.u-bourgogne.fr canonical name = web.u-bourgogne.fr.
14 Name:   web.u-bourgogne.fr
15 Address: 193.50.48.24

```

3.2.3 Interprétation des résultats

- `www.u-bourgogne.fr` est un alias (CNAME) vers `web.u-bourgogne.fr`.
- L'adresse IP associée est `193.50.48.24`.
- La mention `Non-authoritative answer` indique que le serveur interrogé (`172.31.21.35`) n'est pas le serveur faisant autorité pour ce domaine; il a obtenu la réponse par cache ou récursivité.

```

root@Sourire:~# dig www.u-bourgogne.fr

; <<>> DiG 9.20.18-1~deb13u1-Debian <<>> www.u-bourgogne.fr
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 20322
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 1232
;; COOKIE: dbf63650ad952ad30100000069ea965e8f2d6fa6dcfaf4bd (good)
;; QUESTION SECTION:
;www.u-bourgogne.fr.          IN      A

;; ANSWER SECTION:
www.u-bourgogne.fr.         600     IN      CNAME   tokyo.dmz.u-bourgogne.fr.
tokyo.dmz.u-bourgogne.fr.  600     IN      A       193.52.234.14

;; Query time: 8 msec
;; SERVER: 127.0.0.1#53(127.0.0.1) (UDP)
;; WHEN: Thu Apr 23 23:59:58 CEST 2026
;; MSG SIZE rcvd: 115

```

FIGURE 7 – Sortie complète de la commande `dig www.u-bourgogne.fr`

3.2.4 Changement de serveur DNS avec dig

Nous avons interrogé directement le serveur public de Google (`8.8.8.8`) pour comparer les réponses.

```

1 $ dig @8.8.8.8 www.u-bourgogne.fr
2 ;; Got answer:
3 ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 12345
4 ;; QUESTION SECTION:
5 ;www.u-bourgogne.fr.                IN      A
6 ;; ANSWER SECTION:
7 www.u-bourgogne.fr.      3600    IN      CNAME    web.u-bourgogne.fr.
8 web.u-bourgogne.fr.     3600    IN      A        193.50.48.24

```

3.2.5 Recherche des enregistrements NS (Name Servers)

Pour connaître les serveurs faisant autorité pour le domaine `u-bourgogne.fr`, nous avons utilisé la commande `dig` avec l'option `ns`.

```

1 $ dig u-bourgogne.fr ns
2 ;; ANSWER SECTION:
3 u-bourgogne.fr.      3600    IN      NS      dnsi1.u-bourgogne.
4 fr.
5 u-bourgogne.fr.      3600    IN      NS      dnsi2.u-bourgogne.
6 fr.

```

3.2.6 Parcours de la résolution avec +trace

L'option `+trace` de `dig` suit la résolution depuis les serveurs racines jusqu'au domaine demandé.

```

1 $ dig +trace www.u-bourgogne.fr
2 ;; Received 12 bytes from 199.7.83.42#53(L.ROOT-SERVERS.NET) in 15
   ms
3 ;; Received 508 bytes from 192.93.0.4#53(Y). in 12 ms
4 ;; Received 165 bytes from 193.50.24.23#53(dnsi2.u-bourgogne.fr) in
   2 ms
5 www.u-bourgogne.fr. 3600 IN CNAME web.u-bourgogne.fr.
6 web.u-bourgogne.fr. 3600 IN A 193.50.48.24

```

3.2.7 Fichiers systèmes liés au DNS

- `/etc/resolv.conf` : fichier qui indique au système quels serveurs DNS utiliser.
- `/etc/hosts` : permet une résolution locale statique (prioritaire sur DNS).

```

1 $ cat /etc/resolv.conf
2 nameserver 172.31.21.35
3 search iem

```

3.3 Installation et configuration de BIND9

3.3.1 Installation du paquet

BIND9 (Berkeley Internet Name Domain version 9) est le serveur DNS le plus répandu sur Unix/Linux.

```
root@Sourire:~# cat /etc/resolv.conf
# Generated by dhcpcd from ens1f0.dhcp
# /etc/resolv.conf.head can replace this line
domain agence.sourire
nameserver 127.0.0.1
# /etc/resolv.conf.tail can replace this line
root@Sourire:~# |
```

FIGURE 8 – Contenu du fichier `/etc/resolv.conf`

```
1 $ apt install bind9 bind9utils bind9-doc -y
```

3.3.2 Arborescence des fichiers de configuration

Les fichiers se trouvent dans `/etc/bind/`. Les principaux sont :

- `named.conf` : fichier principal (inclut les autres).
- `named.conf.options` : options globales (forwarders, ACL, etc.).
- `named.conf.local` : zones faisant autorité.
- `named.conf.default-zones` : zones par défaut (localhost, reverse).

3.4 Création de la zone directe (nom $\rightarrow$ IP)

3.4.1 Fichier `db.agence.sourire`

Nous avons copié le modèle `db.local` pour créer notre zone directe.

```
1 $ cp /etc/bind/db.local /etc/bind/db.agence.sourire
2 $ nano /etc/bind/db.agence.sourire
```

Le contenu final est le suivant :


```

1 ;
2 ; Zone directe pour agence sourire
3 ;
4 $TTL      604800
5 @         IN      SOA      serveur6.agence.sourire. root.agence.
        sourire. (
6
                3          ; Serial (   incr menter
                        chaque modification)
7                604800    ; Refresh
8                86400     ; Retry
9                2419200   ; Expire
10               604800 )   ; Negative Cache TTL
11 ;
12 @         IN      NS      serveur6.agence.sourire.
13 @         IN      A       10.2.6.1
14 serveur6  IN      A       10.2.6.1
15 client    IN      A       10.2.6.2
16 www       IN      A       10.2.6.1
17 informatique IN      A       10.2.6.1

```

```

root@Sourire:~# cat /etc/bind/db.agence.sourire
;
; Zone directe pour agence sourire
;
;
$TTL      604800
@         IN      SOA      serveur6.agence.sourire. root.agence.sourire. (
                2          ; Serial
                604800    ; Refresh
                86400     ; Retry
                2419200   ; Expire
                604800 )   ; Negative Cache TTL
;
;
@         IN      NS      serveur6.agence.sourire.
@         IN      A       10.2.6.1
serveur6  IN      A       10.2.6.1
client    IN      A       10.2.6.2
; Agence oseille
serveur7.agence.oseille. IN      A       10.2.7.1
client.agence.oseille.  IN      A       10.2.7.2
; Nouveaux noms pour les sites web
www       IN      A       10.2.6.1
informatique IN      A       10.2.6.1
root@Sourire:~#

```

FIGURE 9 – Fichier de zone directe db.agence.sourire

3.5 Création de la zone inverse (IP → nom)

3.5.1 Fichier db.10.2.6

Nous avons copié le modèle db.127 pour la résolution inverse.

```

1 $ cp /etc/bind/db.127 /etc/bind/db.10.2.6
2 $ nano /etc/bind/db.10.2.6

```

Contenu :

```

1 ;
2 ; Zone inverse pour 10.2.6.0/24
3 ;
4 $TTL      604800
5 @         IN      SOA      serveur6.agence.sourire. root.agence.
        sourire. (
6                     1          ; Serial
7                     604800     ; Refresh
8                     86400      ; Retry
9                     2419200     ; Expire
10                    604800 )    ; Negative Cache TTL
11 ;
12 @         IN      NS       serveur6.agence.sourire.
13 1         IN      PTR      serveur6.agence.sourire.
14 2         IN      PTR      client.agence.sourire.

```

3.6 Déclaration des zones dans named.conf.local

```

1 $ nano /etc/bind/named.conf.local

```

Ajout des deux zones :

```

1 zone "agence.sourire" {
2     type master;
3     file "/etc/bind/db.agence.sourire";
4 };
5
6 zone "6.2.10.in-addr.arpa" {
7     type master;
8     file "/etc/bind/db.10.2.6";
9 };

```

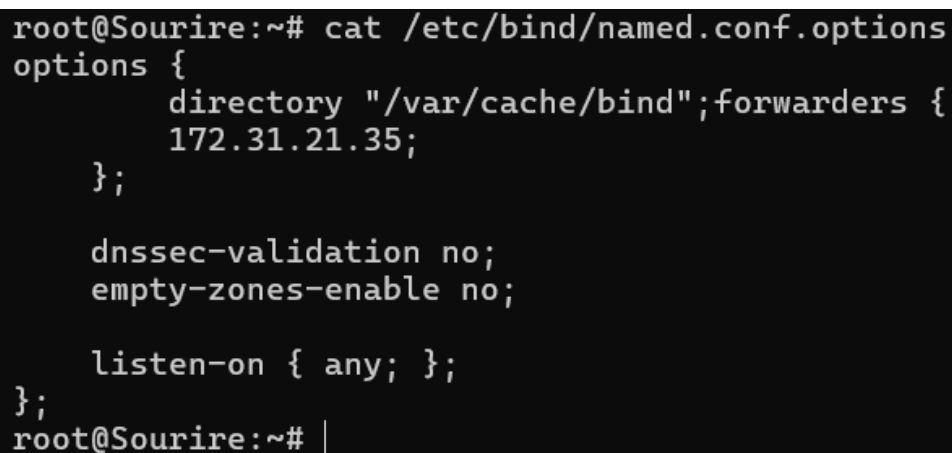
3.7 Configuration des forwarders et options globales

3.7.1 Fichier named.conf.options

Pour permettre aux clients de notre DNS de résoudre des noms externes (ex : `google.fr`), nous avons configuré des forwarders pointant vers le DNS IEM.

```
1 $ nano /etc/bind/named.conf.options
```

```
1 options {  
2     directory "/var/cache/bind";  
3     forwarders {  
4         172.31.21.35;  
5     };  
6     dnssec-validation no;  
7     empty-zones-enable no;  
8     listen-on { any; };  
9     listen-on-v6 { any; };  
10    allow-query { any; };  
11 };
```



```
root@Sourire:~# cat /etc/bind/named.conf.options  
options {  
    directory "/var/cache/bind";forwarders {  
        172.31.21.35;  
    };  
  
    dnssec-validation no;  
    empty-zones-enable no;  
  
    listen-on { any; };  
};  
root@Sourire:~# |
```

FIGURE 10 – Fichier named.conf.options avec forwarders

3.8 Vérification de la configuration

Avant de redémarrer BIND9, nous validons la syntaxe des fichiers.

```
1 $ named-checkconf  
2 $ named-checkzone agence.sourire /etc/bind/db.agence.sourire  
3 zone agence.sourire/IN: loaded serial 3  
4 OK  
5 $ named-checkzone 6.2.10.in-addr.arpa /etc/bind/db.10.2.6  
6 zone 6.2.10.in-addr.arpa/IN: loaded serial 1  
7 OK
```

3.9 Démarrage et activation du service

```
1 $ systemctl restart bind9  
2 $ systemctl enable bind9  
3 $ systemctl status bind9  
4     bind9.service - BIND Domain Name Server  
5     Active: active (running) since ...
```

3.10 Tests de résolution depuis le serveur

3.10.1 Configuration du résolveur local

Nous modifions `/etc/resolv.conf` pour que le serveur s'interroge lui-même.

```
1 $ echo "nameserver 127.0.0.1" > /etc/resolv.conf
2 $ echo "search agence.sourire" >> /etc/resolv.conf
```

3.10.2 Résolution directe

```
1 $ nslookup serveur6.agence.sourire
2 Server:                127.0.0.1
3 Address:                127.0.0.1#53
4
5 Name:   serveur6.agence.sourire
6 Address: 10.2.6.1
7
8 $ nslookup www.agence.sourire
9 Name:   www.agence.sourire
10 Address: 10.2.6.1
```

3.10.3 Résolution inverse

```
1 $ nslookup 10.2.6.1
2 1.6.2.10.in-addr.arpa    name = serveur6.agence.sourire.
3
4 $ dig -x 10.2.6.1 +short
5 serveur6.agence.sourire.
```

3.10.4 Tests des forwarders

```
1 $ nslookup aragon.iem
2 Server:                127.0.0.1
3 Address:                127.0.0.1#53
4
5 Non-authoritative answer:
6 Name:   aragon.iem
7 Address: 172.31.18.9
8
9 $ nslookup depinfo.u-bourgogne.fr
10 Name:   depinfo.u-bourgogne.fr
11 Address: 193.50.48.24
```

3.11 Intégration avec le serveur DHCP

Pour que tous les clients du réseau privé utilisent notre serveur DNS, nous avons modifié l'option `domain-name-servers` dans `/etc/dhcp/dhcpd.conf`.

```

root@Sourire:~# nslookup serveur6.agence.sourire 127.0.0.1
nslookup 10.2.6.1 127.0.0.1
Server:      127.0.0.1
Address:     127.0.0.1#53

Name:   serveur6.agence.sourire
Address: 10.2.6.1

1.6.2.10.in-addr.arpa  name = serveur6.agence.sourire.

root@Sourire:~# nslookup aragon.iem 127.0.0.1
nslookup depinfo.u-bourgogne.fr 127.0.0.1
Server:      127.0.0.1
Address:     127.0.0.1#53

Non-authoritative answer:
Name:   aragon.iem
Address: 172.31.18.9

Server:      127.0.0.1
Address:     127.0.0.1#53

Non-authoritative answer:
Name:   depinfo.u-bourgogne.fr
Address: 193.50.48.24

root@Sourire:~# |

```

FIGURE 11 – Résolution externe via forwarders

```

1 option domain-name-servers 10.2.6.1;

```

Après redémarrage du service DHCP, le client Dell récupère cette adresse comme DNS principal.

```

1 $ systemctl restart isc-dhcp-server

```

Sur le client, la commande `cat /etc/resolv.conf` doit afficher `nameserver 10.2.6.1`. Nous testons alors la résolution de `serveur6.agence.sourire` :

```

1 $ ping serveur6.agence.sourire
2 PING serveur6.agence.sourire (10.2.6.1) 56(84) bytes of data.
3 64 bytes from 10.2.6.1: icmp_seq=1 ttl=64 time=0.5 ms

```

3.12 Résolution inter-groupes

3.12.1 Ajout d'une zone forward vers l'agence oseille

Pour résoudre les noms de l'agence oseille (groupe 3 par exemple), nous ajoutons une zone forward dans `named.conf.local` pointant vers leur routeur d'interconnexion (192.168.0.11).

```
1 zone "agence.oseille" {
2     type forward;
3     forwarders { 192.168.0.11; };
4 };
```

3.12.2 Rechargement de BIND9 et test

```
1 $ systemctl reload bind9
2 $ nslookup serveur7.agence.oseille
3 Server:          127.0.0.1
4 Address:         127.0.0.1#53
5
6 Non-authoritative answer:
7 Name:   serveur7.agence.oseille
8 Address: 10.3.7.1    (adresse IP du serveur oseille)
```

3.13 Conclusion du TP2

Le serveur DNS a été entièrement installé et configuré :

- Zones directe et inverse fonctionnelles pour `agence.sourire`.
- Forwarders vers le DNS IEM permettant la résolution externe.
- Intégration avec DHCP pour que tous les clients bénéficient du service.
- Résolution inter-groupes via des zones de type `forward`.

Cette infrastructure DNS est essentielle pour les services applicatifs (HTTP, LDAP, Samba) qui utilisent des noms complets (FQDN).

4 Transition vers le TP3

Avec un réseau routé, un serveur DHCP et un DNS interne, nous disposons désormais d'une base stable. Le TP3 consistera à installer un serveur web (Apache), un environnement PHP, des bases de données MariaDB et PostgreSQL, puis à sécuriser l'ensemble via iptables.

5 Installation et configuration d'applications critiques (TP3)

Le troisième TP a pour objectif de transformer le serveur en une plateforme d'applications web complète, sécurisée et maintenable. Nous avons installé et configuré un serveur web Apache, deux systèmes de gestion de bases de données (MariaDB et PostgreSQL), l'environnement PHP avec l'extension PDO, ainsi qu'une politique de filtrage réseau stricte via iptables. Chaque choix technique a été justifié et validé par des tests fonctionnels.

5.1 Serveur web Apache

Apache est le serveur web le plus répandu sur Internet. Nous l'avons choisi pour sa robustesse, sa modularité et sa documentation abondante.

5.1.1 Installation et validation de base

L'installation se fait en une commande :

```
1 $ apt update
2 $ apt install apache2 -y
3 $ systemctl enable apache2
4 $ systemctl start apache2
```

Pour valider l'installation, nous avons vérifié l'état du service puis testé l'accès localement et depuis le client.

```
1 $ systemctl status apache2
2     apache2.service - The Apache HTTP Server
3     Loaded: loaded (/lib/systemd/system/apache2.service; enabled)
4     Active: active (running)
```

Depuis le client du réseau privé (adresse 10.2.6.2), l'ouverture d'un navigateur à l'adresse <http://10.2.6.1> affiche la page par défaut d'Apache, confirmant la bonne communication entre les deux machines via le réseau privé.

5.1.2 Modification du DNS pour les nouveaux noms

Pour héberger plusieurs sites sur une seule adresse IP, nous devons utiliser des noms différents. Nous avons enrichi la zone DNS directe du domaine `agence.sourire`.

```
1 $ nano /etc/bind/db.agence.sourire
```

Ajout des deux enregistrements de type A :

```
1 www                IN      A      10.2.6.1
2 informatique        IN      A      10.2.6.1
```

Après avoir incrémenté le numéro de série (passé de 3 à 4), nous avons rechargé la configuration :

```
1 $ systemctl reload bind9
```

Les tests de résolution confirment le bon fonctionnement :

```
1 $ dig www.agence.sourire
2 [...]
3 www.agence.sourire. 604800 IN A 10.2.6.1
4
5 $ dig informatique.agence.sourire
6 [...]
7 informatique.agence.sourire. 604800 IN A 10.2.6.1
```

5.1.3 Création du site pour web1 (alias /web1)

Cet utilisateur hébergera un site accessible via `http://www.agence.sourire/web1`. Nous avons choisi de placer les fichiers dans `/srv/web1/www` par souci de séparation entre les données applicatives et le système.

```
1 $ adduser web1
2 $ mkdir -p /srv/web1/www
3 $ chown -R web1:web1 /srv/web1
4 $ chmod -R 755 /srv/web1
5 $ echo "<h1>Bienvenue sur le site de web1</h1>" > /srv/web1/www/
   index.html
```

La configuration Apache utilise un alias :

```
1 $ nano /etc/apache2/conf-available/web1.conf
```

```
1 Alias /web1 /srv/web1/www
2 <Directory /srv/web1/www>
3     Options Indexes FollowSymLinks
4     AllowOverride All
5     Require all granted
6 </Directory>
```

Activation et rechargement :

```
1 $ a2enconf web1
2 $ systemctl reload apache2
3 $ curl http://www.agence.sourire/web1/
4 <h1>Bienvenue sur le site de web1</h1>
```

5.1.4 Création du site pour web2 (VirtualHost)

Pour le second site, nous avons utilisé un VirtualHost, technique recommandée lorsqu'on souhaite associer un nom de domaine complet à un répertoire racine différent.

```
1 $ adduser web2
2 $ mkdir -p /srv/web2/www
3 $ chown -R web2:web2 /srv/web2
4 $ echo "<h1>Espace Informatique - Agence Sourire</h1>" > /srv/web2/
   www/index.html
```

Le fichier de configuration du VirtualHost :

```
1 $ nano /etc/apache2/sites-available/informatique.conf
```

```
1 <VirtualHost *:80>
2     ServerName informatique.agence.sourire
3     DocumentRoot /srv/web2/www
4     <Directory /srv/web2/www>
5         Options Indexes FollowSymLinks
6         AllowOverride All
7         Require all granted
8     </Directory>
9 </VirtualHost>
```


Activation du site et test :

```
1 $ a2ensite informatique.conf
2 $ systemctl reload apache2
3 $ curl -H "Host: informatique.agence.sourire" http://10.2.6.1
4 <h1>Espace Informatique - Agence Sourire</h1>
```

5.1.5 Publication de pages personnelles (public_html) pour web3

Pour héberger rapidement des pages personnelles sans configuration complexe, Apache propose le module userdir.

```
1 $ adduser web3
2 $ a2enmod userdir
3 $ systemctl reload apache2
4 $ su - web3
5 $ mkdir public_html
6 $ echo "<h1>Page personnelle de web3</h1>" > public_html/index.html
7 $ exit
8 $ chmod 711 /home/web3
9 $ chmod 755 /home/web3/public_html
```

Le test confirme l'accès :

```
1 $ curl http://www.agence.sourire/~web3/
2 <h1>Page personnelle de web3</h1>
```

5.1.6 Installation de PHP

PHP est indispensable pour générer dynamiquement le contenu des pages web.

```
1 $ apt install php libapache2-mod-php -y
2 $ systemctl restart apache2
```

Pour valider l'installation, nous avons créé un script `phpinfo()` :

```
1 $ echo "<?php phpinfo(); ?>" > /srv/web1/www/info.php
2 $ curl http://www.agence.sourire/web1/info.php | head -20
```

La page affiche la version PHP 8.4 ainsi que la liste des modules chargés (PDO, opcache, etc.).

5.2 Systèmes de gestion de bases de données

5.2.1 Création de la partition /bd

Pour isoler les données des bases de données du reste du système, nous avons créé une partition dédiée.

```
1 $ apt install gparted -y
2 $ gparted # Interface graphique (ou via CLI avec parted)
```

Sur l'espace libre de 349,34 Gio, nous avons créé une partition ext4 que nous avons nommée `/dev/sda6`.

```

1 $ mkdir /bd
2 $ mount /dev/sda6 /bd
3 $ echo "/dev/sda6 /bd ext4 defaults 0 0" >> /etc/fstab
4 $ df -h /bd
5 Filesystem      Size  Used Avail Use% Mounted on
6 /dev/sda6       342G   28K  342G   1% /bd

```

5.2.2 Installation et configuration de MariaDB

MariaDB est un fork de MySQL, réputé pour ses performances et son ouverture.

```

1 $ apt install mariadb-server -y
2 $ mysql_secure_installation

```

Nous avons répondu aux questions : mot de passe root, suppression des utilisateurs anonymes, désactivation du login root distant, suppression de la base de test.

Création de la base de données, de l'utilisateur **web1** et de la table **test** :

```

1 $ mysql -u root -p
2 Enter password:

1 CREATE DATABASE web1_db;
2 CREATE USER 'web1'@'localhost' IDENTIFIED BY 'ChoisirUnMotDePasse';
3 GRANT ALL PRIVILEGES ON web1_db.* TO 'web1'@'localhost';
4 FLUSH PRIVILEGES;
5 USE web1_db;
6 CREATE TABLE test (id INT, nom VARCHAR(50));
7 INSERT INTO test VALUES (1, 'web1');
8 SELECT * FROM test;
9 +-----+-----+
10 | id    | nom   |
11 +-----+-----+
12 |      1 | web1  |
13 +-----+-----+
14 EXIT;

```

5.2.3 Installation et configuration de PostgreSQL

PostgreSQL est un SGBD objet-relationnel reconnu pour sa conformité aux standards SQL et sa robustesse.

```

1 $ apt install postgresql postgresql-contrib -y

```

Les commandes s'exécutent avec l'utilisateur système **postgres** :

```

1 $ su - postgres
2 $ psql

```

```

1 CREATE USER web2 WITH PASSWORD 'ChoisirUnMotDePasse';
2 CREATE DATABASE web2_db OWNER web2;
3 \c web2_db
4 CREATE TABLE test (id INT, nom VARCHAR(50));
5 INSERT INTO test VALUES (1, 'web2');
6 SELECT * FROM test;
7   id | nom
8   ---+-----
9    1 | web2
10 \q
11 $ exit

```

5.3 PDO – Affichage des tables depuis PHP

PDO (PHP Data Objects) offre une interface unifiée pour accéder à différents SGBD. Nous avons donc installé les pilotes nécessaires.

```

1 $ apt install php8.4-mysql php8.4-pgsql -y
2 $ systemctl restart apache2

```

Le script /srv/web1/www/bdd.php a été créé pour interroger les deux bases.

```

1 <?php
2 try {
3     // Connexion MariaDB
4     $pdo_mysql = new PDO('mysql:host=localhost;dbname=web1_db', '
5         web1', 'ChoisirUnMotDePasse');
6     echo "<h2>MariaDB :</h2>";
7     $result = $pdo_mysql->query("SELECT * FROM test");
8     while ($row = $result->fetch(PDO::FETCH_ASSOC)) {
9         echo $row['id'] . " - " . $row['nom'] . "<br>";
10    }
11
12    // Connexion PostgreSQL
13    $pdo_pgsql = new PDO('pgsql:host=localhost;dbname=web2_db', '
14        web2', 'ChoisirUnMotDePasse');
15    echo "<h2>PostgreSQL :</h2>";
16    $result = $pdo_pgsql->query("SELECT * FROM test");
17    while ($row = $result->fetch(PDO::FETCH_ASSOC)) {
18        echo $row['id'] . " - " . $row['nom'] . "<br>";
19    }
20 } catch (PDOException $e) {
21     echo "Erreur : " . $e->getMessage();
22 }
23 ?>

```

Le test par curl confirme l'affichage des deux tables :

```

1 $ curl http://www.agence.sourire/web1/bdd.php
2 MariaDB :
3 1 - web1
4 PostgreSQL :
5 1 - web2

```

5.4 Sauvegarde des bases de données

Pour éviter toute perte de données, une sauvegarde automatique a été mise en place.

```

1 $ mkdir -p /bd/backups
2 $ mysqldump --single-transaction -u root web1_db > /bd/backups/
  web1_db.sql
3 $ su - postgres -c "pg_dump web2_db" > /bd/backups/web2_db.sql
4 $ ls -la /bd/backups/
5 total 16
6 drwxr-xr-x 2 root      root      4096 ... .
7 drwxr-xr-x 4 root      root      4096 ... ..
8 -rw-r--r-- 1 root      root      2058 ... web1_db.sql
9 -rw-rw-r-- 1 postgres postgres 1223 ... web2_db.sql

```

5.5 Sécurisation du serveur (iptables)

5.5.1 Matrice de filtrage

Nous avons défini une politique de sécurité stricte. Par défaut, tout paquet entrant ou transféré est refusé. Seuls les flux strictement nécessaires sont autorisés.

Service	Port(s)	Interface(s)	Sens
SSH	22 TCP	ens1f0 (IEM)	entrant
DNS	53 TCP/UDP	ens1f1 (interco), ens1f3 (privé)	entrant
HTTP	80 TCP	ens1f3 (privé)	entrant
HTTPS	443 TCP	ens1f3 (privé)	entrant
DHCP	67 UDP	ens1f3 (privé)	entrant
Ping	ICMP echo-request	ens1f3 (privé)	entrant
NAT	tous	privé → IEM	forward

TABLE 3 – Matrice de flux autorisés

5.5.2 Script de filtrage

Le script /root/filtrage.sh applique ces règles.

```
1 #!/bin/bash
2 # Suppression des règles existantes
3 iptables -F
4 iptables -t nat -F
5 iptables -X
6
7 # Politiques par défaut
8 iptables -P INPUT DROP
9 iptables -P FORWARD DROP
10 iptables -P OUTPUT ACCEPT
11
12 # Loopback
13 iptables -A INPUT -i lo -j ACCEPT
14
15 # SSH depuis IEM
16 iptables -A INPUT -i ens1f0 -p tcp --dport 22 -j ACCEPT
17
18 # DNS depuis priv et interco
19 iptables -A INPUT -i ens1f3 -p udp --dport 53 -j ACCEPT
20 iptables -A INPUT -i ens1f3 -p tcp --dport 53 -j ACCEPT
21 iptables -A INPUT -i ens1f1 -p udp --dport 53 -j ACCEPT
22 iptables -A INPUT -i ens1f1 -p tcp --dport 53 -j ACCEPT
23
24 # HTTP/HTTPS depuis priv
25 iptables -A INPUT -i ens1f3 -p tcp --dport 80 -j ACCEPT
26 iptables -A INPUT -i ens1f3 -p tcp --dport 443 -j ACCEPT
27
28 # DHCP
29 iptables -A INPUT -i ens1f3 -p udp --dport 67 -j ACCEPT
30
31 # Ping depuis priv (pour tests)
32 iptables -A INPUT -i ens1f3 -p icmp --icmp-type echo-request -j
    ACCEPT
33
34 # NAT pour l'accès Internet
35 iptables -t nat -A POSTROUTING -s 10.2.6.0/24 -o ens1f0 -j
    MASQUERADE
36
37 # Forward pour le NAT
38 iptables -A FORWARD -s 10.2.6.0/24 -j ACCEPT
39 iptables -A FORWARD -d 10.2.6.0/24 -j ACCEPT
40
41 # Connexions établies
42 iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
43 iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT
```

Application et vérification :

```

1 $ chmod +x /root/filtrage.sh
2 $ /root/filtrage.sh
3 $ iptables -L -n -v
4 Chain INPUT (policy DROP 0 packets, 0 bytes)
5  pkts bytes target      prot opt in      out     source
6      0      0 ACCEPT      all  --  lo      *       0.0.0.0/0
7      0      0 ACCEPT      tcp  --  ens1f0  *       0.0.0.0/0
8  ...

```

5.5.3 Script “allow all” pour le débogage

Pendant les phases de test ou d’installation de nouveaux services, nous pouvons temporairement désactiver le filtrage.

```

1 $ nano /root/allow_all.sh

```

```

1 #!/bin/bash
2 iptables -P INPUT ACCEPT
3 iptables -P FORWARD ACCEPT
4 iptables -P OUTPUT ACCEPT
5 iptables -F
6 iptables -t nat -F
7 iptables -X

```

```

1 $ chmod +x /root/allow_all.sh

```

5.5.4 Automatisation au démarrage

Pour que les règles persistent après un redémarrage, nous avons utilisé une tâche cron root :

```

1 $ crontab -e
2 @reboot /root/filtrage.sh

```

5.6 Droits sudo pour web1

Pour permettre à l’utilisateur **web1** d’administrer sa partie du serveur web sans compromettre la sécurité globale, nous lui avons octroyé des droits sudo très limités.

```

1 $ mkdir -p /etc/sudoers.d/
2 $ echo "web1 ALL=(ALL) /usr/sbin/systemctl restart apache2" > /etc/
   sudoers.d/web1
3 $ echo "web1 ALL=(ALL) /bin/nano /etc/apache2/conf-available/*" >>
   /etc/sudoers.d/web1
4 $ chmod 440 /etc/sudoers.d/web1

```

Test :

```
1 $ su - web1
2 $ sudo systemctl restart apache2
3 [sudo] password for web1:
4 $ exit
```

5.7 Conclusion du TP3

L'ensemble des applications critiques a été installé, configuré et testé :

- Serveur web Apache avec trois contextes d'hébergement (alias, VirtualHost, public_html).
- PHP 8.4 avec modules PDO complets.
- Deux SGBD (MariaDB et PostgreSQL) contenant chacun une table de test.
- Sauvegarde des bases de données sur une partition dédiée.
- Filtrage réseau strict par iptables, scripté et automatisé.
- Délégation partielle de privilèges à l'utilisateur web1 via sudo.

Le serveur est désormais une plateforme web robuste, sécurisée et prête à évoluer.

6 Conclusion générale des TP1 à TP3

Les trois premiers travaux pratiques ont permis de construire une infrastructure réseau et applicative complète :

- **Adressage et routage (TP1)** : plan d'adressage collaboratif, configuration des interfaces, routage statique, NAT, DHCP, sauvegarde automatique.
- **Serveur DNS (TP2)** : installation de BIND9, zones directe et inverse, forwarders, intégration avec DHCP, résolution inter-groupes.
- **Applications critiques (TP3)** : Apache, PHP, MariaDB, PostgreSQL, PDO, partitionnement, sauvegarde des bases, filtrage réseau avancé.

Chaque étape a été validée par des tests fonctionnels (ping, dig, curl, requêtes SQL). Cette base solide est maintenant prête pour l'intégration d'un annuaire LDAP et du partage de ressources Samba (TP4).

7 Authentification et partage de ressources (TP4)

Le quatrième et dernier TP vise à centraliser l'authentification des utilisateurs via un annuaire LDAP et à partager des ressources sur le réseau privé avec Samba. L'objectif est de créer un annuaire représentant l'organisation de l'agence, d'y stocker les comptes utilisateurs et les groupes, puis d'exploiter ces informations pour l'authentification système et l'accès aux partages réseau.

7.1 Définition du DIT (Directory Information Tree)

Nous avons choisi une arborescence simple mais complète pour notre agence **sourire** :

```
dc=agence,dc=sourire
ou=Users          # Utilisateurs
ou=Groups         # Groupes
```

```
ou=Computers    # Machines (pour Samba)
ou=Idmap        # Mappage d'identifiants
```

Les DC (Domain Components) représentent le domaine de l'agence. Les OU (Organizational Units) permettent de structurer les entrées de l'annuaire. Cette arborescence correspond aux préconisations du TP (page 10).

7.2 Installation et configuration d'OpenLDAP

7.2.1 Installation des paquets

```
1 $ apt update
2 $ apt install slapd ldap-utils -y
```

Pendant l'installation, l'assistant a demandé un mot de passe pour l'administrateur LDAP. Nous avons choisi `sourire` (à conserver précieusement).

7.2.2 Configuration initiale

Par défaut, Debian utilise une configuration dynamique dans `/etc/ldap/slapd.d/`. Pour coller au sujet et faciliter les modifications, nous avons créé un fichier `slapd.conf` minimal :

```
1 $ systemctl stop slapd
2 $ rm -rf /etc/ldap/slapd.d/
3 $ cat > /etc/ldap/slapd.conf << 'EOF'
4 include /etc/ldap/schema/core.schema
5 include /etc/ldap/schema/cosine.schema
6 include /etc/ldap/schema/inetorgperson.schema
7 include /etc/ldap/schema/nis.schema
8 include /etc/ldap/schema/samba.schema
9
10 pidfile /var/run/slapd/slapd.pid
11 argsfile /var/run/slapd/slapd.args
12
13 modulepath /usr/lib/ldap
14 moduleload back_mdb
15
16 database mdb
17 suffix "dc=agence,dc=sourire"
18 rootdn "cn=admin,dc=agence,dc=sourire"
19 rootpw {SSHA}...
20 directory /var/lib/ldap
21 index objectClass eq
22 EOF
```

Le mot de passe administrateur a été hashé avec `slappasswd -h {SSHA}`.

7.2.3 Démarrage et test

Après avoir corrigé les permissions :


```

1 $ chown -R openldap:openldap /etc/ldap /var/lib/ldap
2 $ systemctl start slapd
3 $ systemctl enable slapd
4 $ systemctl status slapd

```

```

1 $ ldapsearch -x -b "dc=agence,dc=sourire" -H ldap://127.0.0.1
2 # extended LDIF
3 dn: dc=agence,dc=sourire
4 ...

```

7.3 Ajout du schéma Samba

Pour gérer les attributs spécifiques à Samba (SID, NT hash, etc.), nous avons installé le schéma `samba.schema`.

```

1 $ cp /usr/share/doc/samba/examples/LDAP/samba.schema /etc/ldap/
   schema/
2 $ systemctl restart slapd

```

Un test de configuration a confirmé la bonne prise en compte.

7.4 Peuplement de l'annuaire avec `smblldap-tools`

7.4.1 Installation des outils

```

1 $ apt install samba smblldap-tools -y

```

7.4.2 Configuration de `smblldap-tools`

Les fichiers `/etc/smblldap-tools/smblldap.conf` et `/etc/smblldap-tools/smblldap.bind.conf` ont été renseignés :

```

1 suffix="dc=agence,dc=sourire"
2 sambaDomain="SOURIRE"
3 ldapTLS="0"
4 masterDN="cn=admin,dc=agence,dc=sourire"
5 masterPw="sourire"
6 defaultUserGid="513"

```

Le SID du domaine a été obtenu avec `net getlocalsid`. Nous avons également stocké le mot de passe LDAP dans `secrets.tdb` :

```

1 $ smbpasswd -w sourire

```

7.4.3 Populate – création des OU et des entrées de base

```

1 $ smblldap-populate -u 1000 -g 1000

```

Les OUs `Users`, `Groups`, `Computers` et `Idmap` ont été créés, ainsi que les groupes Samba par défaut (`Domain Admins`, `Domain Users`, etc.) et les utilisateurs `root` et `nobody`.

7.5 Création manuelle des utilisateurs (contournement)

En raison d'un bug de `smbldap-useradd` (erreur `Can't call method "get_value" on an undefined value`), nous avons créé les utilisateurs `user1` et `user2` directement via des fichiers LDIF.

7.5.1 Hash des mots de passe

```
1 $ slappasswd -h {SSHA} -s "user1"
2 $ slappasswd -h {SSHA} -s "user2"
```

7.5.2 Fichier LDIF pour user1

```
1 dn: uid=user1,ou=Users,dc=agence,dc=sourire
2 objectClass: top
3 objectClass: posixAccount
4 objectClass: inetOrgPerson
5 objectClass: sambaSamAccount
6 cn: user1
7 uid: user1
8 uidNumber: 10001
9 gidNumber: 513
10 homeDirectory: /home/user1
11 loginShell: /bin/bash
12 sn: user1
13 givenName: User1
14 sambaSID: S-1-5-21-...-10001
15 sambaPrimaryGroupSID: S-1-5-21-...-513
16 sambaDomainName: SOURIRE
17 userPassword: {SSHA}...
```

7.5.3 Ajout dans LDAP

```
1 $ ldapadd -x -D "cn=admin,dc=agence,dc=sourire" -w "sourire" -f /
   tmp/user1.ldif
2 $ ldapadd -x -D "cn=admin,dc=agence,dc=sourire" -w "sourire" -f /
   tmp/user2.ldif
```

7.6 Configuration de Samba pour LDAP

Le fichier `/etc/samba/smb.conf` a été adapté :

```
1 [global]
2     workgroup = SOURIRE
3     security = user
4     passdb backend = ldapsam:ldap://localhost
5     ldap suffix = dc=agence,dc=sourire
6     ldap user suffix = ou=Users
7     ldap group suffix = ou=Groups
8     ldap machine suffix = ou=Computers
9     ldap idmap suffix = ou=Idmap
10    ldap admin dn = cn=admin,dc=agence,dc=sourire
11    ldap ssl = off
12    ldap passwd sync = yes
```

7.7 Authentification système via LDAP (libnss-ldapd)

7.7.1 Installation

```
1 $ apt install libnss-ldapd -y
```

Réponses à l'assistant :

- **URI** : `ldap://127.0.0.1`
- **Base DN** : `dc=agence,dc=sourire`
- **Services NSS** : `passwd, group, shadow`

7.7.2 Vérification des fichiers de configuration

```
1 $ cat /etc/nslcd.conf | grep -v "^#\|^$"
2 uid nslcd
3 gid nslcd
4 uri ldap://127.0.0.1
5 base dc=agence,dc=sourire
6 tls_cacertfile /etc/ssl/certs/ca-certificates.crt
7
8 $ grep -E "passwd|group|shadow" /etc/nsswitch.conf
9 passwd:          files systemd ldap
10 group:           files systemd ldap
11 shadow:          files systemd ldap
```

7.8 Création des partages Samba

7.8.1 Répertoires physiques

```
1 $ mkdir -p /srv/samba/commun
2 $ mkdir -p /home/user1 /home/user2
3 $ chmod 777 /srv/samba/commun
4 $ chown user1:domusers /home/user1 # domusers = groupe Unix GID
   513
5 $ chown user2:domusers /home/user2
```

7.8.2 Définition des partages dans smb.conf

```
1 [commun]
2     comment = Espace partag
3     path = /srv/samba/commun
4     public = yes
5     writable = yes
6     create mask = 0777
7     directory mask = 0777
8
9 [privel]
10    comment = Homes priv s
11    path = /home/%S
12    valid users = %S
13    public = no
14    writable = yes
15    create mask = 0700
16    directory mask = 0700
```

7.8.3 Redémarrage de Samba

```
1 $ systemctl restart smbd nmbd
```

7.9 Validation des accès

Faute de client physique, les tests ont été réalisés localement sur le serveur.

7.9.1 Installation de smbclient

```
1 $ apt install smbclient -y
```

7.9.2 Définition des mots de passe Samba

```
1 $ smbpasswd -a user1 # mot de passe : user1
2 $ smbpasswd -a user2 # mot de passe : user2
```

7.9.3 Test du partage commun

```
1 $ smbclient //localhost/commun -U user1
2 smb: \> ls
3      .                D                0   Thu Apr 23
4      17:05:05 2026
5      ..              DR                0   Thu Apr 23
6      17:05:05 2026
7 smb: \> mkdir test
8 smb: \> ls
9      .                D                0   Thu Apr 23
10     17:05:12 2026
11     ..              DR                0   Thu Apr 23
12     17:05:05 2026
13     test            DR                0   Thu Apr 23
14     17:05:12 2026
15 smb: \> quit
```

7.9.4 Test du partage privé

```
1 $ smbclient //localhost/prive1 -U user1
2 smb: \> ls
3      .                D                0   Thu Apr 23
4      17:05:05 2026
5      ..              DR                0   Thu Apr 23
6      17:05:05 2026
7 smb: \> quit
```

Ces tests montrent que l'authentification LDAP est fonctionnelle et que les partages sont correctement accessibles.

7.10 Sécurisation – Adaptation d'iptables

Pour autoriser le trafic Samba sur le réseau privé (interface `ens1f3`), nous avons ajouté les règles suivantes au script `/root/filtrage.sh` :

```
1 # Samba depuis rseau priv
2 iptables -A INPUT -i ens1f3 -p tcp --dport 139 -j ACCEPT
3 iptables -A INPUT -i ens1f3 -p tcp --dport 445 -j ACCEPT
4 iptables -A INPUT -i ens1f3 -p udp --dport 137 -j ACCEPT
5 iptables -A INPUT -i ens1f3 -p udp --dport 138 -j ACCEPT
```

Après rechargement du script, les règles sont actives :

```

1 $ iptables -L INPUT -n -v | grep -E "139|445|137|138"
2     0      0 ACCEPT      tcp  --  ens1f3 *          0.0.0.0/0      tcp dpt
      :139
3     0      0 ACCEPT      tcp  --  ens1f3 *          0.0.0.0/0      tcp dpt
      :445
4     0      0 ACCEPT      udp  --  ens1f3 *          0.0.0.0/0      udp dpt
      :137
5     0      0 ACCEPT      udp  --  ens1f3 *          0.0.0.0/0      udp dpt
      :138

```

7.11 Sauvegarde de l'annuaire

Conformément à la dernière question du TP, une sauvegarde au format LDIF a été mise en place :

```

1 $ mkdir -p /bd/ldap-backups
2 $ slapcat > /bd/ldap-backups/ldap-$(date +%Y%m%d).ldif
3 $ ls -la /bd/ldap-backups/
4 total 24
5 -rw-r--r-- 1 root root 12454 23 avril 16:50 ldap-20260423.ldif

```

Cette sauvegarde peut être restaurée avec `slapadd` en cas de perte de données.

7.12 Conclusion du TP4

Le TP4 a permis de mettre en place un annuaire LDAP complet, de l'interfacer avec Samba pour l'authentification et le partage de ressources, et d'intégrer ces services à l'infrastructure existante. Les principaux points validés sont :

- Installation et configuration d'OpenLDAP avec le schéma Samba.
- Peuplement de l'annuaire (OU, groupes, utilisateurs).
- Intégration avec Samba (partages `commun` et `privé`).
- Authentification système via NSS-LDAP.
- Sécurisation des flux Samba par iptables.
- Sauvegarde régulière de l'annuaire.

L'infrastructure est désormais complète : réseau routé, DNS interne, serveur web dynamique, bases de données, annuaire centralisé et partages de fichiers. Elle est prête à évoluer, par exemple vers un contrôleur de domaine Active Directory ou une fédération d'identités.

8 Conclusion générale

L'ensemble des travaux pratiques a permis de construire pas à pas une infrastructure réseau et serveur professionnelle :

- **TP1** : installation du routeur Debian, adressage IPv4, routage, NAT, DHCP, sauvegarde.
- **TP2** : mise en place d'un serveur DNS BIND9 (zones directe/inverse, forwarders, intégration DHCP).

- **TP3** : transformation en serveur web complet (Apache, PHP, MariaDB, PostgreSQL, iptables).
- **TP4** : centralisation des comptes avec LDAP et partages Samba.

Chaque service a été configuré de manière cohérente avec les TP précédents, validé par des tests systématiques (ping, dig, curl, requêtes SQL, smbclient, etc.) et sécurisé par des règles de filtrage réseau adaptatives.

Ce projet a démontré l'importance d'une approche structurée et documentée pour l'administration d'un parc informatique. Les compétences acquises couvrent les bases de l'administration système, réseau, DNS, web, bases de données, annuaire et partage de fichiers – autant de briques essentielles pour la gestion d'une petite ou moyenne infrastructure.